OpenMP Tasks: Parallel Quicksort

Lesson 12, second example: `task` on a recursive, irregular algorithm

Gustavo Lozano

November 2026

Source: van der Pas, Stotzer & Terboven [1], chapter on tasking. Prerequisite: `openmp_tasks.pdf` (Lesson 12).

1 Why this example

Fibonacci showed the mechanics of tasks, but its call tree is regular: the work could be predicted in advance. Quicksort is the opposite. Each step picks a pivot, moves the smaller values to the left and the larger ones to the right (the *partition*), and then sorts both sides. How big each side is depends on the data and is only known after the partition:

- the work is a **tree**, not a loop: `omp for` cannot express it;
- the pieces have **unpredictable sizes**: no static split can balance them.

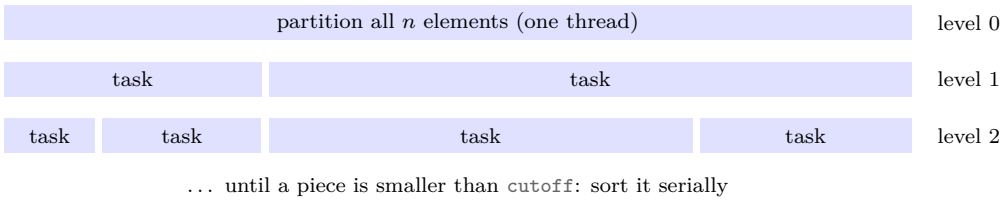


Figure 1: Each partition splits its range in two pieces of unpredictable size; each piece becomes a task.

Intuition: who does what, and when. Figure 2 follows an example with $n = 100$ elements and `cutoff` $= 40$, run by 2 threads. To keep the numbers simple, assume a partition of m elements costs m time units and a serial sort of m elements costs $3m$.

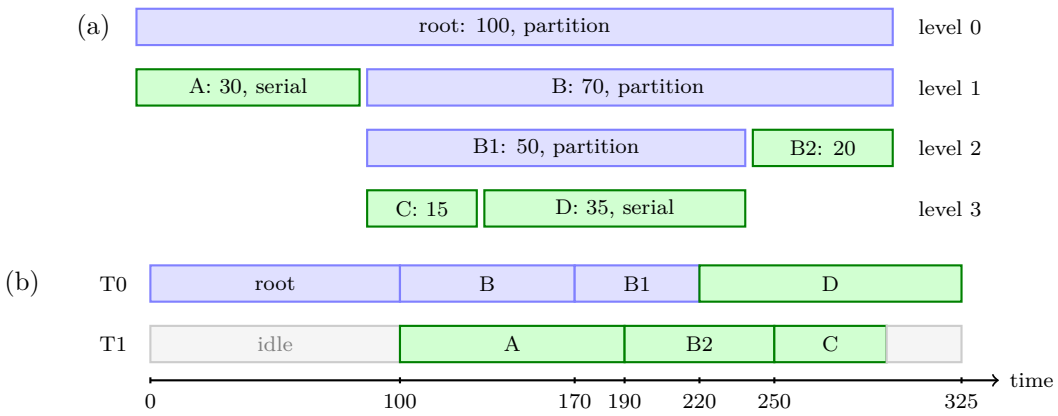


Figure 2: (a) Tasks and their sizes; blue: partition and create two tasks, green: below the cutoff, sorted serially. (b) A possible timeline with 2 threads: a thread that becomes free takes the next task in the queue.

What Figure 2 shows:

- The sizes 30/70, then 50/20, then 15/35 are only known after each partition; nobody could have assigned the work in advance.
- No thread is told what to do: T1 takes A because it is free at time 100; it takes B2 at 190 and C at 250 for the same reason.
- The run takes 325 time units instead of 520 serially ($100 + 70 + 50$ for partitions, $3 \cdot (30 + 20 + 15 + 35)$ for serial sorts): a speed-up of 1.6 with 2 threads.
- T1 is idle at the start, because only one piece exists until the first partition ends, and at the end, because D is the largest last task. These two effects limit the speed-up (see the experiment).

2 Quicksort by hand (one thread)

The array to sort, with its indices above each element ($n = 8$):

$a =$

0	1	2	3	4	5	6	7
7	2	9	4	1	8	5	3

We follow the call (Listing 1), with `OMP_NUM_THREADS=1`:

```
quicksort_tasks(a, lo = 0, hi = 7, cutoff = 3)
```

With one thread, tasks wait in a queue and T0 runs them in creation order.

pivot candidates | pivot | range in work | final position

Stage 1 call `(0,7)`: `hi-lo` $= 7 \geq$ `cutoff` $= 3 \Rightarrow$ partition and create two tasks

1a. Pivot: median of three. `mid` $= 0 + (7 - 0)/2 = 3$.

Test	Action	$a[0], \ldots, a[7]$							
start	—	7	2	9	4	1	8	5	3
$a[3] = 4 < a[0] = 7$	`swap(a, 3, 0)`	**4**	2	9	**7**	1	8	5	3
$a[7] = 3 < a[0] = 4$	`swap(a, 7, 0)`	**3**	2	9	7	1	8	5	**4**
$a[7] = 4 < a[3] = 7$	`swap(a, 7, 3)`	3	2	9	**4**	1	8	5	**7**
$3 \le 4 \le 7$	`swap(a, 3, 7)`: median to the end	3	2	9	**7**	1	8	5	**4**

1b. Scan with pivot $= 4$. `i` = next slot for a value $<$ pivot; `j` visits $a[0..6]$. Green: values already known to be $<$ pivot.

`j`	`i`	$a[j] <$ pivot	Action	$a[0], \ldots, a[7]$							
0	0	true	`swap(a, 0, 0)`	3	2	9	7	1	8	5	4
1	1	true	`swap(a, 1, 1)`	3	2	9	7	1	8	5	4
2	2	false	—	3	2	9	7	1	8	5	4
3	2	false	—	3	2	9	7	1	8	5	4
4	2	true	`swap(a, 2, 4)`	3	2	**1**	7	**9**	8	5	4
5	3	false	—	3	2	1	7	9	8	5	4
6	3	false	—	3	2	1	7	9	8	5	4

1c. Place the pivot and create tasks. Swap $a[\texttt{i} = 3]$ with the pivot; `partition` returns $p = 3$. Then two tasks are created and the call returns.

0	1	2	3	4	5	6	7
3	2	1	**4**	9	8	5	**7**

queue: **A** = (0,2)
B = (4,7)

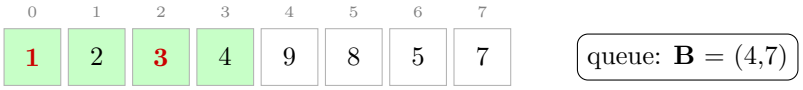
Stage 2 task A `(0,2)`: `hi-lo` $= 2 <$ `cutoff` $= 3 \Rightarrow$ `quicksort_serial(a,0,2)`

Loop of `quicksort_serial(a, 0, 2)` (lines 51–55 of Listing 1):

iter.	`lo`	`hi`	`lo < hi`	`p`	`p-lo < hi-p`	recursive call	update
1	0	2	true	1	1 < 1: false	`quicksort_serial(a, 2, 2)`	`hi = p-1 = 0`
2	0	0	false	—	—	—	exit loop

`quicksort_serial(a, 2, 2)` has one element: `2 < 2` is false, it returns at once.
Iteration 1 calls `partition(a, 0, 2)`, `mid` $= 1$:

Step	Action	$a[0], a[1], a[2]$		
start	—	3	2	1
$a[1] = 2 < a[0] = 3$	`swap(a, 1, 0)`	**2**	**3**	1
$a[2] = 1 < a[0] = 2$	`swap(a, 2, 0)`	**1**	3	**2**
$a[2] = 2 < a[1] = 3$	`swap(a, 2, 1)`	1	**2**	**3**
median to the end	`swap(a, 1, 2)`; pivot $= 2$	1	**3**	**2**
$j = 0,\ i = 0$: $a[0] = 1 <$ pivot $= 2$	`swap(a, 0, 0)`	1	3	2
$j = 1,\ i = 1$: $a[1] = 3 \ge$ pivot $= 2$	—	1	3	2
place pivot, $p = 1$	`swap(a, 1, 2)`	1	**2**	**3**



Stage 3 task B (4,7): hi-lo $= 3 \geq$ cutoff $= 3 \Rightarrow$ partition and create two tasks

`partition(a,4,7)`, `mid` $= 5$:

Step	Action	$a[4], \ldots, a[7]$
start	—	9 8 5 7
$a[5] = 8 < a[4] = 9$	`swap(a, 5, 4)`	8 9 5 7
$a[7] = 7 < a[4] = 8$	`swap(a, 7, 4)`	7 9 5 8
$a[7] = 8 < a[5] = 9$	`swap(a, 7, 5)`	7 8 5 9
median to the end	`swap(a, 5, 7)`; pivot $= 8$	7 9 5 8
$j = 4,\ i = 4$: $a[4] = 7 <$ pivot $= 8$	`swap(a, 4, 4)`	7 9 5 8
$j = 5,\ i = 5$: $a[5] = 9 \geq$ pivot $= 8$	—	7 9 5 8
$j = 6,\ i = 5$: $a[6] = 5 <$ pivot $= 8$	`swap(a, 5, 6)`	7 5 9 8
place pivot, $p = 6$	`swap(a, 6, 7)`	7 5 8 9



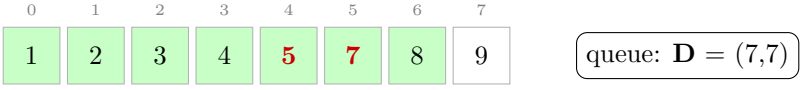
Stage 4 task C (4,5): hi-lo $= 1 <$ cutoff $= 3 \Rightarrow$ quicksort_serial(a,4,5)

Loop of `quicksort_serial(a, 4, 5)` (lines 51–55 of Listing 1):

iter.	lo	hi	lo < hi	p	p-lo < hi-p	recursive call	update
1	4	5	true	4	$0 < 1$: true	`quicksort_serial(a, 4, 3)`	`lo = p+1 = 5`
2	5	5	false	—	—	—	exit loop

`quicksort_serial(a, 4, 3)` is an empty range: `4 < 3` is false, it returns at once.
Iteration 1 calls `partition(a, 4, 5)`, `mid` $= 4$ (same cell as `lo`):

Step	Action	$a[4], a[5]$
start	—	7 5
$a[5] = 5 < a[4] = 7$	`swap(a, 5, 4)`	5 7
median to the end	`swap(a, 4, 5)`; pivot $= 5$	7 5
$j = 4,\ i = 4$: $a[4] = 7 \geq$ pivot $= 5$	—	7 5
place pivot, $p = 4$	`swap(a, 4, 5)`	5 7

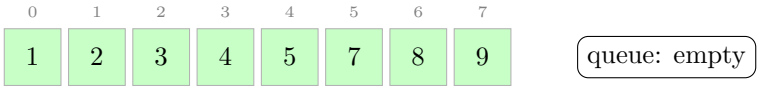


Stage 5 task D (7,7): hi-lo $= 0 <$ cutoff $= 3 \Rightarrow$ quicksort_serial(a,7,7)

Loop of `quicksort_serial(a, 7, 7)` (lines 51–55 of Listing 1):

iter.	lo	hi	lo < hi	p	p-lo < hi-p	recursive call	update
1	7	7	false	—	—	—	exit loop

The queue is empty; T0 leaves the barrier.



3 The code

```
/*
 * quicksort_tasks.c -- OpenMP Lesson 12 (second example): parallel quicksort
 * with tasks.
 * Source: van der Pas, Stotzer & Terboven (2017), "Using OpenMP -- The Next
 *         Step", chapter on tasking.
 *
 * Quicksort splits an array around a pivot into two parts of UNPREDICTABLE
 * size, then sorts each part. The parts can be sorted independently, so each
 * one becomes a task. No loop describes this work: its shape is only known
 * while the program runs.
 *
 * Compile:
 *     gcc -Wall -O2 -fopenmp -o quicksort_tasks quicksort_tasks.c
 * Run:   ./quicksort_tasks n cutoff
 *     OMP_NUM_THREADS=4 ./quicksort_tasks 20000000 10000
 */

#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define MAXT 64

/* Work done by each thread, in elements sorted in its serial pieces.
   Padded to one 64-byte cache line per thread: no false sharing (Lesson 8). */
typedef struct { _Alignas(64) long elements; long pieces; } counter_t;
static counter_t work[MAXT];

static void swap(double *a, long i, long j) { double t = a[i]; a[i] = a[j]; a[j] = t
    ; }

/* Partition a[lo..hi] around the median of a[lo], a[mid], a[hi].
   Returns p such that a[lo..p-1] <= a[p] <= a[p+1..hi].                    */
static long partition(double *a, long lo, long hi)
{
    long mid = lo + (hi - lo) / 2;
    if (a[mid] < a[lo]) swap(a, mid, lo);
    if (a[hi]  < a[lo]) swap(a, hi, lo);
    if (a[hi]  < a[mid]) swap(a, hi, mid);
    swap(a, mid, hi);                       /* median goes to the end */
    double pivot = a[hi];
    long i = lo;
    for (long j = lo; j < hi; j++)
        if (a[j] < pivot) swap(a, i++, j);
    swap(a, i, hi);
    return i;
```

```
46 }
47
48 /* plain recursive quicksort, used for small pieces */
49 static void quicksort_serial(double *a, long lo, long hi)
50 {
51     while (lo < hi) {
52         long p = partition(a, lo, hi);
53         if (p - lo < hi - p) { quicksort_serial(a, lo, p - 1); lo = p + 1; }
54         else                 { quicksort_serial(a, p + 1, hi); hi = p - 1; }
55     }
56 }
57
58 static void quicksort_tasks(double *a, long lo, long hi, long cutoff)
59 {
60     if (hi - lo < cutoff) {                 /* small piece: not worth a task */
61         quicksort_serial(a, lo, hi);
62         int t = omp_get_thread_num();
63         work[t].elements += hi - lo + 1;    /* only thread t writes work[t] */
64         work[t].pieces++;
65         return;
66     }
67     long p = partition(a, lo, hi);          /* split: sizes known only now */
68
69     /* Two independent sub-problems -> two tasks. Variables are firstprivate
70        in a task: each task gets a COPY of the pointer a, but the copy points
71        to the same shared array, so the sorting happens in place.          */
72     #pragma omp task
73     quicksort_tasks(a, lo, p - 1, cutoff);
74     #pragma omp task
75     quicksort_tasks(a, p + 1, hi, cutoff);
76
77     /* No taskwait: nothing below needs the children's results. All tasks
78        are finished at the barrier at the end of the parallel region.     */
79 }
80
81 static int is_sorted(const double *a, long n)
82 {
83     for (long i = 1; i < n; i++)
84         if (a[i - 1] > a[i]) return 0;
85     return 1;
86 }
87
88 int main(int argc, char **argv)
89 {
90     long n      = (argc > 1) ? atol(argv[1]) : 20000000;
91     long cutoff = (argc > 2) ? atol(argv[2]) : 10000;
92     int  T      = omp_get_max_threads();
93     if (T > MAXT) { fprintf(stderr, "at most %d threads\n", MAXT); return 1; }
94
95     double *a = malloc((size_t)n * sizeof(double));
96     double *b = malloc((size_t)n * sizeof(double));
97     if (!a || !b) { fprintf(stderr, "out of memory\n"); return 1; }
98     srand(42);
99     for (long i = 0; i < n; i++) a[i] = b[i] = (double)rand() / RAND_MAX;
100
101    /* serial reference time */
102    double t = omp_get_wtime();
103    quicksort_serial(b, 0, n - 1);
104    double t_serial = omp_get_wtime() - t;
```

```
105
106     /* parallel sort of the same data */
107     t = omp_get_wtime();
108     #pragma omp parallel
109     {
110         #pragma omp single              /* one thread starts the recursion */
111         quicksort_tasks(a, 0, n - 1, cutoff);
112     }                                   /* barrier: every task is done */
113     double t_tasks = omp_get_wtime() - t;
114
115     printf("n = %ld, cutoff = %ld, threads = %d\n", n, cutoff, T);
116     printf("serial : %.3f s\n", t_serial);
117     printf("tasks  : %.3f s   speed-up %.2f   sorted: %s\n",
118            t_tasks, t_serial / t_tasks, is_sorted(a, n) ? "yes" : "NO");
119     printf("work per thread (elements sorted in serial pieces):\n");
120     for (int i = 0; i < T; i++)
121         printf("  thread %2d: %10ld elements in %6ld pieces (%5.1f%%)\n", i,
122                work[i].elements, work[i].pieces, 100.0 * work[i].elements / n);
123
124     free(a); free(b);
125     return 0;
126 }
```

Listing 1: **`quicksort_tasks.c`** – parallel quicksort with tasks.

```
gcc -Wall -O2 -fopenmp -o quicksort_tasks quicksort_tasks.c
OMP_NUM_THREADS=2 ./quicksort_tasks 20000000 10000
```

Output on a 2-core machine (your times will differ):

```
n = 20000000, cutoff = 10000, threads = 2
serial : 2.534 s
tasks  : 1.289 s   speed-up 1.97   sorted: yes
work per thread (elements sorted in serial pieces):
  thread  0:    9305664 elements in   1577 pieces ( 46.5%)
  thread  1:   10690906 elements in   1854 pieces ( 53.5%)
```

The percentages add up to slightly less than 100% because the pivots are placed by the partitions, not by the serial pieces.

Key ideas

- Tasks fit work whose shape appears while it runs: trees, recursion, irregular sizes.
- Load balance comes for free: a thread that finishes a small piece takes the next task; nobody decides the distribution in advance.
- `taskwait` only where results are needed; the barrier of the region collects the rest.
- Tasks that write disjoint parts of a shared array need no synchronization.

4 Experiments

Experiment 1: work distribution and speed-up

Experiment

Question: how well does the irregular work spread over the threads, and what limits the speed-up?

1. Run with an increasing number of threads:

```
for t in 1 2 4 $(nproc); do
    OMP_NUM_THREADS=$t ./quicksort_tasks 20000000 10000
done
```

2. For each run, note the speed-up and the smallest and largest share of work in the per-thread table.

What to observe and answer.

1. The threads sort different numbers of pieces and elements, yet the speed-up stays close to the number of threads. Why does unequal *work* not mean idle threads?
2. Run the same command twice. Does the distribution of work change? Why?
3. The first partition runs on one thread, before any task exists. Why does it limit the speed-up on machines with many cores?

Answers (read after trying). (1) Every thread keeps taking tasks until none are left, so all threads stay busy until the end. A thread that happened to draw small pieces simply takes more of them. Unequal shares only show that the pieces had different sizes, not that anyone waited. (With 8 threads on 2 million elements, the shares ranged from 8% to 16.5%.)
(2) Yes. Which thread takes which task depends on timing, which changes every run. The *result* does not change, because each piece is sorted correctly by whoever takes it.
(3) Before the first partition finishes there is only one piece and only one thread can work; the second level offers only two pieces, and so on. On the test machine the first partition alone took 0.042 s of the 2.43 s serial sort, and the chain of partitions down to the largest piece adds more. This serial part (Amdahl's law) caps the speed-up at roughly $2.43/0.12 \approx 20$ on this input, no matter how many cores. Parallelizing the partition itself removes this limit, at the cost of a more complex algorithm.

Experiment 2: choosing the cutoff

Experiment

Question: which `cutoff` gives the best time, and why?

1. Time `quicksort_tasks` for cutoffs from 10^2 to n with all your cores (Listing 2):

```
sh run_cutoff.sh 20000000 $(nproc)
```

   It writes `cutoff.dat`: plain text, one line per cutoff, columns separated by spaces (1: cutoff, 2: serial time, 3: tasks time, 4: speed-up); lines starting with `#` are comments. Any plotter can read it (gnuplot, Python `numpy.loadtxt`, a spreadsheet).

2. Plot it with gnuplot (Listing 3), which writes `cutoff.png`:

```
gnuplot plot_cutoff.gp
```

What to observe and answer.

1. What happens to the speed-up when the cutoff is close to n? And at `cutoff` $= n$?
2. Does a very small cutoff (100) hurt on your machine? Why could it?
3. Which range of cutoffs is good, and how would you choose one for a different n or number of threads?

```
#!/bin/sh
# run_cutoff.sh -- time quicksort_tasks for several cutoffs and write cutoff.dat
#
# Usage:   sh run_cutoff.sh [n] [threads]
#          sh run_cutoff.sh 20000000 $(nproc)
#
# cutoff.dat: plain text, one line per cutoff, columns separated by spaces
#   1 cutoff   2 serial time (s)   3 tasks time (s)   4 speed-up
# Lines starting with # are comments (ignored by gnuplot, numpy.loadtxt, ...).

N=${1:-20000000}
T=${2:-$(nproc)}

echo "# n = $N, threads = $T"           > cutoff.dat
echo "# cutoff serial_s tasks_s speedup" >> cutoff.dat
for c in 100 300 1000 3000 10000 30000 100000 300000 1000000 3000000 10000000 $N; do
    OMP_NUM_THREADS=$T ./quicksort_tasks $N $c |
        awk -v c=$c '/^serial/ {s = $3} /^tasks/ {print c, s, $3, $6}'
done >> cutoff.dat
cat cutoff.dat
```

Listing 2: **`run_cutoff.sh`** – time the sort for several cutoffs and write `cutoff.dat`.

```
# plot_cutoff.gp -- plot cutoff.dat (written by run_cutoff.sh)
#
# Usage:   gnuplot plot_cutoff.gp      -> cutoff.png
# Columns of cutoff.dat: 1 cutoff, 2 serial time, 3 tasks time, 4 speed-up

set terminal pngcairo size 800,700 font ",11"
set output 'cutoff.png'

set logscale x
set format x "10^{%L}"
set xrange [50:*]
set grid
# first line of cutoff.dat is "# n = ..., threads = ...": use it as the title
info = system("sed -n '1s/^# //p' cutoff.dat")
set multiplot layout 2,1 title info

set ylabel 'time (s)'
set yrange [0:*]
set key bottom left
plot 'cutoff.dat' using 1:3 with linespoints pt 7 title 'tasks', \
     ''           using 1:2 with linespoints pt 6 dt 2 title 'serial'

set xlabel 'cutoff (elements)'
```

```
24  set ylabel 'speed-up'
25  set yrange [0:*]
26  set key bottom left
27  plot 'cutoff.dat' using 1:4 with linespoints pt 5 title 'serial / tasks'
28
29  unset multiplot
```

Listing 3: **`plot_cutoff.gp`** – gnuplot script: time and speed-up against the cutoff.

Answers (read after trying). Figure 3 shows two runs on the author's laptop with $n = 2 \cdot 10^7$: 2 threads and 8 threads.

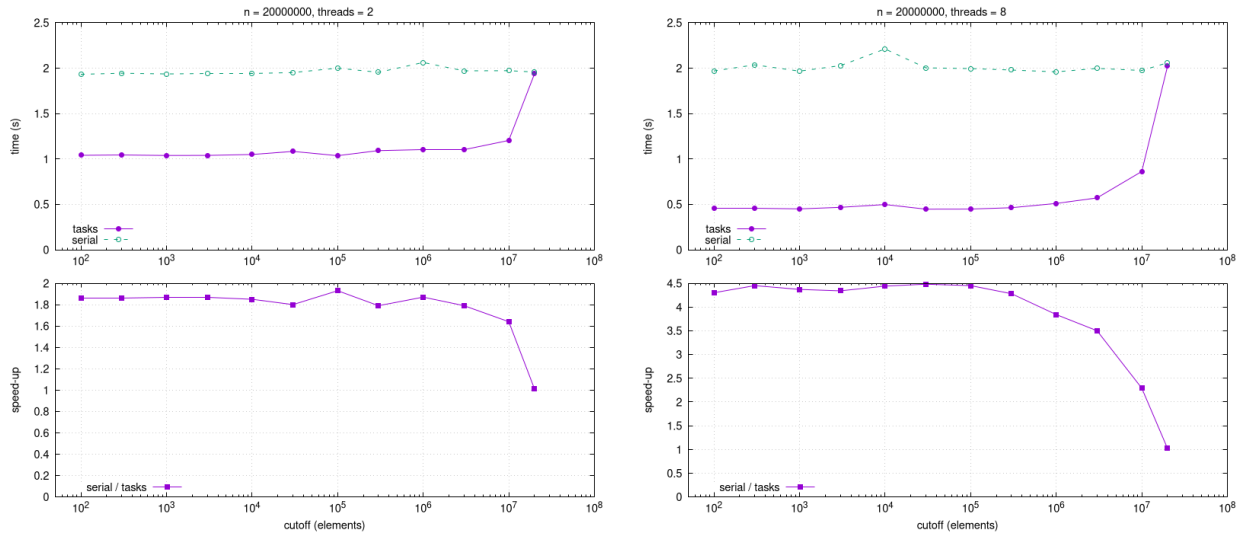


Figure 3: `cutoff.png` for $n = 2 \cdot 10^7$: 2 threads (left), 8 threads (right).

(1) With a large cutoff only a few pieces become tasks: about $n/$`cutoff` serial pieces, of unequal size, so some thread finishes late. With 2 threads this only shows at `cutoff` $= 10^7$ (2 pieces, speed-up 1.6). With 8 threads the drop starts earlier, because 8 threads need more pieces: 3.8 at 10^6, 2.3 at 10^7. At `cutoff` $= n$ the very first call is already below the cutoff and sorts everything serially: no task is created, and the speed-up is 1.
(2) Each task costs some time to create and schedule (about a microsecond, Lesson 12). A cutoff of 100 creates hundreds of thousands of tasks. With 2 threads this costs nothing visible (1.86, against the best 1.93); with 8 threads it costs a little (4.3, against the best 4.5). The overhead grows with the number of tasks and threads.
(3) The flat part of the curve: from 10^2 to about $3 \cdot 10^6$ with 2 threads, but only to about 10^5 with 8 threads: more threads move the right edge to the left. A safe choice keeps both conditions true: many more pieces than threads ($n/$`cutoff` $\gg$ threads, e.g. at least 100 per thread) and pieces large enough to hide the task overhead (thousands of elements). For $n = 2 \cdot 10^7$ and 8 threads, a cutoff around 10^4 satisfies both.

Next lesson. `taskloop` and task dependencies (`depend`).

References

[1] R. van der Pas, E. Stotzer, and C. Terboven. *Using OpenMP — The Next Step: Affinity, Accelerators, Tasking, and SIMD*. MIT Press, Cambridge, MA, 2017.